

Bektur Akkabakov

CS4412: Advanced Algorithms

Project 2: Convex Hull

Report

1.

procedure ConvexHull(Q):

Input: Q is a set of points in 2D space.

Output: H is a set of points of computed convex hull.

// base case

if(len(Q)<=1): return Q

// recursive calls

recursion depth: $\log_2(n)$ where $n = |Q|$

leftSH = ConvexHull(Q[0, [|Q|/2]])

rightSH = ConvexHull(Q[$\lceil \frac{|Q|}{2} \rceil$, |Q|])

// variables preparation

O(n)

rPoint = find rightmost point of leftSH

lPoint = find leftmost point of rightSH

rNext = find next to the rightmost point of leftSH in **counterclockwise** direction

lNext = find next to the leftmost point of rightSH in **clockwise** direction

// upper common tangent computation

// O(n) - at the worst-case loops together will iterate over each point in two sub-hulls, i.e. $2 \cdot n/2 = n$

while slope(rPoint, lPoint) > slope(rNext, lPoint) or slope(rPoint, lPoint) < slope(rPoint, lNext):

while slope(rPoint, lPoint) > slope(rNext, lPoint):

rPoint = rNext

rNext = find next to the rightmost point of leftSH in **counterclockwise** direction

while slope(rPoint, lPoint) < slope(rPoint, lNext):

lPoint = lNext

lNext = find next to the leftmost point of rightSH in **clockwise** direction

// saving vertices of upper common tangent

upperR = rPoint

upperL = lPoint

// variables preparation

O(n)

rPoint = find rightmost point of leftSH

lPoint = find leftmost point of rightSH

rNext = find next to the rightmost point of leftSH in **clockwise** direction

lNext = find next to the leftmost point of rightSH in **counterclockwise** direction

// lower common tangent computation

O(n)

while slope(lPoint, rPoint) < slope(lPoint, rNext) or slope(lPoint, rPoint) > slope(lNext, rPoint):

while slope(lPoint, rPoint) < slope(lPoint, rNext):

rPoint = rNext

rNext = find next to the rightmost point of leftSH in **clockwise** direction

while slope(lPoint, rPoint) > slope(lNext, rPoint):

lPoint = lNext

lNext = find next to the leftmost point of rightSH in **counterclockwise** direction

```

// saving vertices of lower common tangent
lowerR = rPoint
lowerL = lPoint

H = []

// construct merged hull in clockwise order
// O(n) algorithm needs to iterate over all n points of merged hull
Add to H all points of rightSH from upperL to lowerL in clockwise order.
Add to H all points of leftSH from lowerR to upperR in clockwise order.

return H

```

2.

ConvexHull is divide and conquer algorithm which breaks down the problem into 2 subproblems of size $n/2$ with combining complexity of $O(n)$, where n is the number of points. Thus, the Master Theorem can be utilized to compute total complexity of the algorithm:

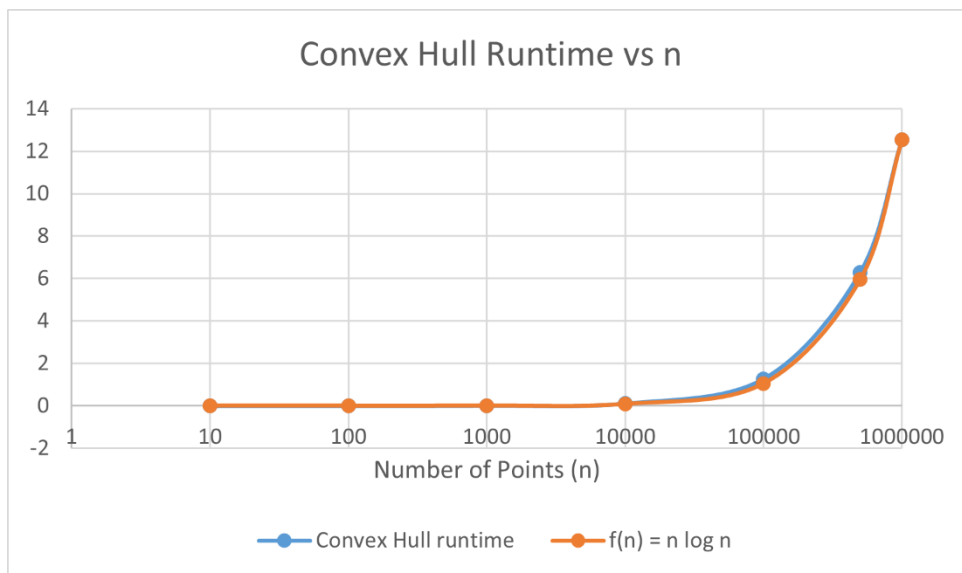
Since $a = 2$, $b = 2$, and $d = 1$, this is case 2 of the Master Theorem

$$T(n) = O(n * \log(n))$$

Each recursive call uses constant extra space. So, recursion stack space is $O(\log n)$. During merge, algorithm creates a new hull list H . In the worst case (all points on hull) $|H|=n$. Note that at lower recursion levels, the temporary hull lists are reused as the recursion returns, so we do not multiply n by the recursion depth. Thus, merging requires $O(n)$ space. Overall total space complexity will be:

$$T(n) = n + \log n = O(n)$$

4.



*The x-axis was plotted on a logarithmic scale to better visualize growth across multiple orders of magnitude (from 10 to 1,000,000). Using a logarithmic scale compresses large values of n and spreads out smaller values, preventing the smaller input sizes from being visually negligible.

5.

Theoretically from point 2, my Convex Hull algorithm should have runtime that grows proportionally to $n \log n$. After scaling $n \log n$ by a constant factor $k \approx 6.29 \times 10^{-7}$ the fitted curve closely matches the measured mean runtime values. This confirms the theoretical complexity $O(n \log n)$.

The value of k, constant of proportionality, was obtained by measuring the factor by which $n \log n$ function is bigger for input size of 1M: $\frac{12.5281}{19931568.57} \approx 6.29 * 10^{-7}$.

6.

From the plotted curves, we observe that for $n=100,000$ and $n=500,000$, the measured mean runtime is slightly higher than the predicted $k * n \log n$ values. This difference is likely due to system overhead such as memory allocation, cache effects, recursion overhead, and other factors. Since Big-O describes asymptotic behavior, small deviations at finite input sizes are expected. Overall, the growth trend still follows $n \log n$ which supports the theoretical analysis.